Programmation Python — fondamentaux et applications

Informatique — Licence 2 (L2)

Cours INF-L2-02

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours

Passer du pseudo-code au vrai code Python : syntaxe et structures de contrôle, fonctions et modules, collections, fichiers et exceptions, programmation orientée objet, bibliothèques requests et pandas, jusqu'à l'automatisation de tâches réelles.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Syntaxe Python : variables, types et structures de contrôle
• Leçon 2 — Fonctions, modules et bibliothèque standard
• Leçon 3 — Listes, tuples, dictionnaires et ensembles
• Leçon 4 — Manipulation de fichiers et gestion des erreurs
• Leçon 5 — Programmation orientée objet en Python
• Leçon 6 — Bibliothèques utiles : requests et pandas en initiation
• Leçon 7 — Projet : automatiser une tâche réelle avec un script Python
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

En première année, tu as appris à raisonner avec l'algorithmique et à traduire tes idées en langage C. Python reprend exactement les mêmes concepts — variables, conditions, boucles, tableaux, fonctions — mais avec une syntaxe allégée qui laisse toute la place au raisonnement : pas de point-virgule obligatoire, pas de déclaration de type, et des blocs délimités par la seule indentation. C'est le langage de référence pour découvrir sérieusement la programmation moderne, de l'automatisation quotidienne à l'analyse de données. Ce cours suppose que tu as déjà pratiqué l'algorithmique et un premier langage : nous n'y reviendrons que brièvement, pour consacrer l'essentiel du temps à ce que Python change concrètement dans ta façon de programmer.

Ce cours ne se contente pas des bases : il vise le niveau attendu en fin de Licence 2, c'est-à-dire être capable d'écrire un programme utile de bout en bout. Tu y apprendras à organiser ton code en fonctions et en modules, à choisir la bonne structure de données, à lire et écrire des fichiers sans faire planter ton programme à la première erreur de saisie, puis à modéliser des objets du monde réel avec des classes. Chaque notion est présentée avec des exemples courts, exécutés et vérifiés un par un, dont tu connais donc le résultat exact avant même de les recopier : à toi ensuite de les modifier et de prévoir l'effet du changement avant de relancer le programme.

La dernière partie ouvre sur l'écosystème qui fait la force de Python : des bibliothèques prêtes à l'emploi comme requests, pour dialoguer avec le Web, et pandas, pour analyser des tableaux de données en quelques lignes. Le projet final assemble tout : tu

écriras un script qui automatise une tâche réelle et répétitive, du genre de celles qu'un technicien ou un gestionnaire fait à la main chaque semaine. Chaque exemple de ce cours a été exécuté et vérifié : les résultats annoncés sont ceux que tu obtiendras. Travaille le cours avec un éditeur ouvert à côté du document : lire du code ne suffit jamais, c'est en le tapant, en le cassant et en le réparant qu'il devient vraiment tien.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Écrire des programmes Python corrects en respectant l'indentation et le typage dynamique.
- Structurer un programme en fonctions réutilisables, avec paramètres, valeurs par défaut et valeurs de retour.
- Choisir et manipuler la bonne collection : liste, tuple, dictionnaire ou ensemble.
- Lire et écrire des fichiers texte et CSV, et protéger son code avec try/except.
- Modéliser un problème simple avec des classes, des attributs, des méthodes et l'héritage.
- Utiliser requests pour interroger un service web et pandas pour analyser un tableau de données.
- Concevoir et livrer un script d'automatisation complet, testé sur un cas réel.

Leçon 1 — Syntaxe Python : variables, types et structures de contrôle

La première chose qui frappe en Python, c'est ce qui a disparu : plus d'accolades, plus de points-virgules obligatoires, plus de déclaration de type avant d'utiliser une variable. L'indentation — le décalage des lignes, en général quatre espaces — remplace les accolades et délimite les blocs. Elle n'est pas décorative : une ligne mal décalée change le sens du programme ou provoque une erreur immédiate. En échange de cette rigueur visuelle, le code devient lisible presque comme du pseudo-code, ce qui était précisément l'intention du langage. Un programme Python se lit donc de haut en bas, comme une suite d'ordres donnés à un assistant méticuleux : chaque ligne compte, chaque décalage a un sens, et l'interpréteur signale immédiatement la ligne qu'il ne comprend pas.

Python est à typage dynamique : une variable reçoit son type de la valeur qu'on lui affecte, et la fonction type() permet de le vérifier à tout moment. Les types de base sont int (entiers), float (nombres à virgule), str (chaînes de caractères) et bool (True ou False). Deux conséquences pratiques dominent le quotidien : input() renvoie toujours une chaîne, qu'il faut convertir avec int() ou float() avant tout calcul ; et les chaînes formatées, les f-strings comme f"Résultat : {x}", sont la manière moderne et propre d'afficher des valeurs mélangées à du texte. Retiens aussi que Python distingue strictement les nombres des chaînes : additionner 2 et "3" est une erreur, là où d'autres langages bricolent une conversion silencieuse qui cache les bugs au lieu de les révéler.

Les structures de contrôle traduisent presque mot pour mot celles de l'algorithmique : if / elif / else pour les décisions, sans parenthèses obligatoires autour de la condition et avec deux-points en fin de ligne. La boucle for parcourt directement les éléments d'une collection — for n in notes — sans compteur manuel dans la plupart des cas ; range() fabrique des suites d'entiers quand on a besoin d'un compteur. La boucle while répète tant qu'une condition reste vraie, et break comme continue gardent leur rôle habituel : sortir de la boucle, ou passer au tour suivant. Les opérateurs de comparaison sont ceux des mathématiques (==, !=, <, >, <=, >=) et se combinent avec and, or et not pour exprimer des conditions composées, exactement comme les ET, OU et NON de l'algorithmique de première année.

L'essentiel
Python se lit comme du pseudo-code : indentation obligatoire pour les blocs, typage dynamique, input() toujours en chaîne. if/elif/else décident, for parcourt les collections, while répète sous condition.

Exemple — Types, mention et parcours d'une liste de notes

```
nom = "Aminata"
age = 21
moyenne = 13.75
admis = moyenne >= 10
print(type(nom), type(age), type(moyenne), type(admis))
# <class 'str'> <class 'int'> <class 'float'> <class 'bool'>

if moyenne >= 16:
mention = "Très bien"
elif moyenne >= 14:
mention = "Bien"
elif moyenne >= 10:
mention = "Passable"
else:
mention = "Ajourné"
print(mention) # Passable

notes = [12, 15, 9, 17, 14]
total = 0
for n in notes:
total += n
print("Total :", total, "| Moyenne :", total / len(notes))
# Total : 67 | Moyenne : 13.4
```

À retenir

- L'indentation fait partie de la syntaxe : un mauvais décalage est une erreur ou un bug, jamais un détail esthétique.
- input() renvoie toujours une str : convertir avec int() ou float() avant de calculer.
- for parcourt directement les valeurs d'une collection ; range() sert quand on veut un compteur.

Exercice

Écris une fonction somme_pairs(nombres) qui reçoit une liste d'entiers et retourne la somme des seuls nombres pairs. Teste-la sur [1, 2, 3, 4, 5, 6] et donne le résultat attendu.

✔ Corrigé
def somme_pairs(nombres): on initialise s = 0, puis on parcourt la liste avec for n in nombres ; à chaque valeur paire (test n % 2 == 0, le reste de la division par 2), on ajoute n à s ; on retourne s à la fin. Sur [1, 2, 3, 4, 5, 6], les pairs sont 2, 4 et 6, donc le résultat vérifié par exécution est 12. Version experte en une ligne, une fois la boucle maîtrisée : return sum(n for n in nombres if n % 2 == 0) donne exactement le même résultat.

Leçon 2 — Fonctions, modules et bibliothèque standard

Une fonction se définit avec le mot-clé def, un nom, des paramètres entre parenthèses et un bloc indenté. Elle rend son résultat avec return ; sans return, elle renvoie la valeur spéciale None. Les paramètres peuvent recevoir des valeurs par défaut — def prix_ttc(prix_ht, taux=0.18) — ce qui rend certains arguments facultatifs à l'appel. Une fonction peut aussi retourner plusieurs valeurs d'un coup, en réalité un tuple que l'on « déballe » à la réception : mini, maxi, moy = statistiques(notes). Bien nommer ses fonctions et leur donner une seule responsabilité reste la règle d'or déjà vue en algorithmique. Une fonction bien découpée se teste isolément : on lui donne des valeurs connues, on vérifie le résultat, et seulement ensuite on l'emploie dans le programme complet, ce qui localise les erreurs au lieu de les chercher partout à la fois.

La portée des variables suit une règle simple : une variable créée dans une fonction y reste locale et disparaît à la fin de l'appel ; une variable du programme principal n'est lisible dans la fonction que si on ne cherche pas à la modifier. Pour documenter une fonction, on place juste sous sa définition une chaîne entre triples guillemets, la docstring, qui explique ce qu'elle fait, ce qu'elle reçoit et ce qu'elle retourne. Cette habitude, qui coûte dix secondes, transforme un code incompréhensible trois mois plus tard en code professionnel. En pratique, on évite presque toujours de modifier une variable globale depuis une fonction : on lui passe les données en paramètres et on récupère le résultat par return, ce qui garde chaque fonction indépendante et prévisible.

Python arrive avec une bibliothèque standard immense, organisée en modules que l'on importe avec import ou from ... import. Le module math fournit sqrt, pi, les fonctions trigonométriques ; datetime manipule les dates ; random tire des nombres au hasard ; os et pathlib dialoguent avec le système de fichiers ; json lit et écrit le format d'échange des applications web. Avant d'écrire toi-même un utilitaire, cherche s'il n'existe pas déjà dans la bibliothèque standard : c'est du code testé par des millions d'utilisateurs. Pour les bibliothèques externes, l'outil pip les installe en une commande. Savoir chercher dans la documentation officielle fait partie du métier : la bibliothèque standard y est décrite fonction par fonction, avec des exemples, et la consulter avant de coder est un réflexe qui fait gagner des heures.

L'essentiel
def + return structurent le code ; valeurs par défaut et retours multiples simplifient les appels. La bibliothèque standard (math, datetime, random, json…) couvre déjà la majorité des besoins courants.

Exemple — Fonctions avec valeur par défaut, retour multiple et modules

```
import math
from datetime import date

def prix_ttc(prix_ht, taux=0.18):
return prix_ht * (1 + taux)

def statistiques(notes):
return min(notes), max(notes), sum(notes) / len(notes)

print(prix_ttc(10000)) # 11800.0
print(prix_ttc(10000, 0.10)) # 11000.0
mini, maxi, moy = statistiques([11, 14, 8, 19])
print(mini, maxi, moy) # 8 19 13.0
print(math.sqrt(144)) # 12.0
print(date(2026, 10, 12).strftime("%d/%m/%Y"))
# 12/10/2026
```

À retenir

- Une fonction sans return renvoie None : si ton calcul « disparaît », cherche un return oublié.
- Paramètre avec valeur par défaut = argument facultatif ; l'appel prix_ttc(10000) utilise le taux 0.18.
- import module donne accès à module.fonction ; from module import fonction importe le nom directement.

Exercice

Écris une fonction mot_de_passe_valide(mdp) qui retourne True si le mot de passe fait au moins 8 caractères, contient au moins un chiffre et au moins un signe de ponctuation, et False sinon. Vérifie avec "iam2026!" et "iam".

✔ Corrigé
On combine trois tests : len(mdp) >= 8 ; any(c.isdigit() for c in mdp) qui vérifie qu'au moins un caractère est un chiffre ; any(c in string.punctuation for c in mdp) pour la ponctuation, après avoir importé le module string. Le corrigé complet retourne l'AND des trois conditions. Résultat vérifié par exécution : mot_de_passe_valide("iam2026!") vaut True (9 caractères, chiffres et « ! »), mot_de_passe_valide("iam") vaut False (trop court, ni chiffre ni ponctuation).

Leçon 3 — Listes, tuples, dictionnaires et ensembles

La liste est la collection de travail de Python : ordonnée, modifiable, capable de mélanger les types même si l'usage courant regroupe des valeurs homogènes. On y accède par indice à partir de 0, avec les indices négatifs qui partent de la fin (liste[-1] est le dernier élément), et les tranches liste[1:3] extraient des sous-listes. Les méthodes essentielles sont append (ajouter à la fin), insert, remove, pop (retirer et récupérer), sort et reverse. Contrairement au C, pas de taille à déclarer ni de débordement à craindre : la liste grandit toute seule, et c'est la fonction qui parcourt qui doit connaître la logique, pas la capacité.

Le tuple ressemble à une liste entre parenthèses, mais il est immuable : une fois créé, on ne peut ni ajouter ni modifier ses éléments. Cette rigidité est une qualité quand les données ne doivent pas changer — des coordonnées géographiques, les jours de la semaine, ou les valeurs multiples retournées par une fonction. Le dictionnaire, lui, range les données par couples clé-valeur : fiche["nom"] plutôt que fiche[0]. Les clés sont le plus souvent des chaînes ; on ajoute un couple par simple affectation, on lit en sécurité avec get(cle, valeur_par_defaut) qui évite l'erreur de clé absente, et on parcourt clés, valeurs ou couples avec keys(), values() et items().

L'ensemble (set) stocke des éléments uniques, sans ordre garanti : en y versant une liste, les doublons disparaissent d'eux-mêmes. Il excelle pour tester l'appartenance (in, très rapide) et pour les opérations ensemblistes héritées des mathématiques discrètes : union (|), intersection (&), différence (−). Enfin, les compréhensions offrent une écriture compacte et très « pythonique » pour construire une collection à partir d'une autre : [x * x for x in range(1, 6)] fabrique les carrés de 1 à 5, et {n: n * 2 for n in [1, 2, 3]} un dictionnaire de doubles. À utiliser quand elles restent lisibles en un coup d'œil.

L'essentiel
Liste = modifiable et ordonnée ; tuple = immuable ; dictionnaire = accès par clé nommée ; ensemble = éléments uniques et opérations ensemblistes. Les compréhensions construisent une collection en une ligne lisible.

Exemple — Les quatre collections en action

```
etudiants = ["Awa", "Moussa", "Fatou"]
etudiants.append("Ibrahim")
etudiants[0] = "Awa D."
print(etudiants) # ['Awa D.', 'Moussa', 'Fatou', 'Ibrahim']
print(etudiants[-1]) # Ibrahim

point = (14.6, -17.4) # tuple : coordonnees figées
print(point[0], point[1]) # 14.6 -17.4

fiche = {"nom": "Moussa", "filiere": "Informatique",
"moyenne": 15.5}
fiche["niveau"] = "L2"
print(fiche.get("age", "non renseigné"))
# non renseigné (pas d'erreur malgré la clé absente)

langages = {"Python", "Java", "Python", "HTML"}
print(sorted(langages)) # ['HTML', 'Java', 'Python']
print(len(langages)) # 3 : le doublon a disparu
```

À retenir

- Choisir la collection selon le besoin : ordre et modification → liste ; données figées → tuple ; accès par nom → dictionnaire ; unicité → ensemble.
- dict.get(cle, defaut) évite l'erreur KeyError quand la clé peut manquer.
- Un doublon dans un ensemble disparaît silencieusement : parfait pour dédoublonner, dangereux si les répétitions comptaient.

Exercice

Écris une fonction compter_mots(phrase) qui retourne un dictionnaire donnant, pour chaque mot de la phrase (en minuscules), le nombre de fois où il apparaît. Applique-la à "le chat voit le chien et le chat dort" et donne le dictionnaire obtenu.

✔ Corrigé

On parcourt phrase.lower().split(), qui découpe la phrase en mots après l'avoir mise en minuscules ; pour chaque mot, compte[mot] = compte.get(mot, 0) + 1 crée le compteur à 1 au premier passage puis l'incrémente. Résultat vérifié par exécution : {'le': 3, 'chat': 2, 'voit': 1, 'chien': 1, 'et': 1, 'dort': 1}. Le patron get(cle, 0) + 1 est l'idiome standard de comptage en Python ; la bibliothèque standard offre aussi collections.Counter qui fait exactement ce travail.

Leçon 4 — Manipulation de fichiers et gestion des erreurs

Un programme qui ne garde rien entre deux exécutions a une mémoire de poisson rouge : les fichiers donnent la persistance. La fonction open(chemin, mode) ouvre un fichier : "r" pour lire (le fichier doit exister), "w" pour écrire en écrasant tout, "a" pour ajouter à la fin. La bonne pratique moderne est l'instruction with open(...) as f, qui ferme le fichier automatiquement à la fin du bloc, même si une erreur survient au milieu. On précise toujours l'encodage, encoding="utf-8", pour que les accents de nos

textes français soient écrits et relus sans surprise. Un chemin de fichier peut être absolu, depuis la racine du disque, ou relatif au dossier depuis lequel le programme est lancé : la plupart des surprises de débutant — « fichier introuvable » alors qu'il est juste là — viennent d'un chemin relatif mal compris.

Pour lire, trois styles coexistent : f.read() charge tout le contenu d'un coup (à réserver aux petits fichiers), f.readlines() donne la liste des lignes, et la boucle for ligne in f traite ligne par ligne sans charger tout le fichier en mémoire — c'est le style à privilégier. Chaque ligne lue conserve son caractère de fin "\n", d'où l'usage constant de strip() pour le retirer. Pour écrire, f.write(texte) ajoute exactement ce qu'on lui donne, sauts de ligne compris ou non : c'est au programmeur de les mettre. Les formats structurés courants — CSV pour les tableaux, JSON pour les échanges — disposent de modules dédiés, csv et json, qui évitent de découper les lignes à la main.

Tôt ou tard, un fichier manque, un disque est plein, un utilisateur tape « abc » là où un nombre est attendu : le programme qui n'a rien prévu s'arrête net avec un message d'erreur. Le bloc try / except intercepte ces exceptions : le code risqué va dans try, le traitement en cas de problème dans except, avec le type d'erreur visé (ValueError, FileNotFoundError, ZeroDivisionError…) pour ne pas masquer les autres problèmes. Le bloc else s'exécute si tout s'est bien passé, et finally s'exécute dans tous les cas — typiquement pour fermer une ressource. On peut aussi lever sa propre erreur avec raise ValueError("message") quand une valeur est inacceptable. La philosophie Python assume qu'il vaut mieux essayer et gérer l'échec que multiplier les tests préventifs.

L'essentiel
with open(...) ferme le fichier à coup sûr ; modes r/w/a ; for ligne in f pour lire sans tout charger. try/except intercepte les erreurs précises plutôt que de laisser le programme planter.

Exemple — Écrire puis relire un fichier, et lire un nombre sans planter

```
with open("notes_iam.txt", "w", encoding="utf-8") as f:
f.write("Awa;15\n")
f.write("Moussa;12\n")
f.write("Fatou;17\n")

with open("notes_iam.txt", "r", encoding="utf-8") as f:
lignes = f.readlines()
print(lignes)
# ['Awa;15\n', 'Moussa;12\n', 'Fatou;17\n']

def note_sure(texte):
try:
valeur = float(texte)
except ValueError:
return None
if 0 <= valeur <= 20:
return valeur
return None

print(note_sure("14.5")) # 14.5
print(note_sure("abc")) # None : pas de plantage
print(note_sure("25")) # None : hors limites
```

À retenir

- "w" écrase le fichier existant sans demander : pour conserver l'historique, utiliser "a".
- Attraper l'exception précise (ValueError…) plutôt qu'un except nu qui cacherait tous les bugs.
- Une ligne lue garde son "\n" : penser à strip() avant de convertir ou de comparer.

Exercice

Écris une fonction moyenne_fichier(chemin) qui lit un fichier contenant un nombre par ligne, ignore proprement les lignes invalides en affichant un avertissement, et retourne la moyenne des nombres valides (ou None s'il n'y en a aucun). Teste-la sur un fichier contenant 10, 15, « erreur », 20.

✔ Corrigé
On ouvre le fichier avec with, on parcourt chaque ligne, et on tente float(ligne.strip()) dans un try : en cas de succès, la valeur rejoint la liste ; en cas de ValueError, on affiche « Ligne ignorée : erreur ». À la fin, si la liste est vide on retourne None, sinon sum(valeurs) / len(valeurs). Résultat vérifié par exécution sur le fichier de test : l'avertissement s'affiche pour la ligne fautive et la fonction retourne 15.0, la moyenne de 10, 15 et 20. C'est le patron « traiter ce qui est traitable, signaler le reste » que l'on retrouve dans tous les scripts d'automatisation sérieux.

Leçon 5 — Programmation orientée objet en Python

Jusqu'ici, nos programmes séparent les données (des variables, des listes) et les traitements (des fonctions) qui les manipulent de l'extérieur. La programmation

orientée objet inverse la perspective : on regroupe dans une même entité — l'objet — ses données, appelées attributs, et les opérations qui le concernent, appelées méthodes. Une classe est le plan de fabrication de ces objets : class Etudiant décrit ce qu'est un étudiant en général ; chaque objet créé à partir d'elle, comme awa ou moussa, est une instance avec ses propres valeurs. On crée un objet en « appelant » la classe comme une fonction : e = Etudiant("Fatou", "Informatique").

La méthode spéciale __init__, le constructeur, s'exécute automatiquement à la création de chaque objet : c'est là que l'on initialise ses attributs avec self.attribut = valeur. Le paramètre self, présent en première position de chaque méthode, désigne l'objet lui-même en train d'être manipulé : self.nom est le nom de CET étudiant-ci. Les autres méthodes décrivent les comportements : ajouter_note, moyenne, presenter. D'autres méthodes spéciales donnent des super-pouvoirs aux objets : __str__ définit ce qu'affiche print(objet), __len__ ce que vaut len(objet), __eq__ comment deux objets se comparent. Ces noms entourés de doubles underscores sont les points d'accroche officiels du langage. À la création, l'objet reçoit ses attributs dans l'ordre des paramètres de __init__ : Etudiant("Fatou", "Informatique") range "Fatou" dans self.nom et "Informatique" dans self.filiere, et chaque objet possède ensuite sa propre copie de ces attributs, indépendante de celle des autres.

L'héritage permet de spécialiser une classe sans la recopier : class EtudiantBoursier(Etudiant) hérite de tous les attributs et méthodes d'Etudiant, ajoute les siens (un montant de bourse) et peut redéfinir ceux qu'il veut adapter. Dans son __init__, on appelle super().__init__(...) pour laisser la classe mère initialiser sa partie. La convention d'encapsulation est légère en Python : un attribut dont le nom commence par un underscore, comme _solde, signale « usage interne, ne touchez pas directement » — le langage ne l'interdit pas, mais la communauté respecte le signal. L'objet bien conçu protège sa cohérence : on modifie un compte via deposer() et retirer(), pas en tripotant son solde.

L'essentiel
Une classe est un plan, un objet est une instance avec ses attributs (données) et ses méthodes (comportements). __init__ initialise, self désigne l'objet courant, l'héritage spécialise sans recopier.

Exemple — Une classe Etudiant complète, puis une spécialisation par héritage

```
class Etudiant:
def __init__(self, nom, filiere):
self.nom = nom
self.filiere = filiere
self.notes = []

def ajouter_note(self, note):
self.notes.append(note)

def moyenne(self):
if not self.notes:
return 0
return sum(self.notes) / len(self.notes)

def presenter(self):
return (f"{self.nom} ({self.filiere}) - "
f"moyenne : {self.moyenne():.2f}")

e1 = Etudiant("Fatou Ndiaye", "Informatique")
e1.ajouter_note(14)
e1.ajouter_note(16)
e1.ajouter_note(13)
print(e1.presenter())
# Fatou Ndiaye (Informatique) - moyenne : 14.33

class EtudiantBoursier(Etudiant):
def __init__(self, nom, filiere, montant):
super().__init__(nom, filiere)
self.montant = montant

e2 = EtudiantBoursier("Sekou Traoré", "Informatique", 50000)
e2.ajouter_note(18)
print(e2.presenter(), "| Bourse :", e2.montant)
# Sekou Traoré (Informatique) - moyenne : 18.00
# | Bourse : 50000
```

À retenir

• self n'est pas un mot-clé magique : c'est le premier paramètre de chaque méthode, qui reçoit l'objet courant automatiquement.

• __init__ ne crée pas l'objet, il l'initialise : les attributs naissent là, avec des valeurs cohérentes dès le départ.

• Hériter, c'est réutiliser puis spécialiser : super().__init__ laisse la classe mère faire sa part du travail.

Exercice

Écris une classe CompteBancaire avec un titulaire et un solde (0 par défaut), des méthodes deposer(montant) et retirer(montant) — retirer retourne True si l'opération est possible, False sinon, sans jamais laisser le solde devenir négatif — et une méthode __str__ qui décrit le compte. Teste : dépôt de 5 000 sur un compte ouvert avec 20 000, retrait de 30 000 refusé, retrait de 10 000 accepté.

✔ Corrigé
Le constructeur range le titulaire et le solde ; deposer n'ajoute que si montant > 0 ; retirer teste 0 < montant <= self.solde avant de soustraire et retourne le booléen correspondant ; __str__ retourne une f-string « Compte de {titulaire} : {solde} FCFA ». Résultats vérifiés par exécution : le retrait de 30 000 retourne False (solde insuffisant), celui de 10 000 retourne True, et l'affichage final donne « Compte de Awa : 15000 FCFA ». Le solde n'a jamais été modifié directement de l'extérieur : c'est l'encapsulation en pratique.

Leçon 6 — Bibliothèques utiles : requests et pandas en initiation

La vraie puissance de Python tient à son écosystème : pour presque chaque besoin courant, quelqu'un a déjà publié une bibliothèque testée, installable en une commande pip install nom_bibliotheque. La règle professionnelle n'est pas de tout réinventer, mais de savoir trouver la bonne bibliothèque, lire sa documentation et l'utiliser correctement. Deux d'entre elles couvrent une grande partie des besoins d'un étudiant de Licence 2 : requests pour communiquer avec le Web, et pandas pour analyser des données tabulaires. Ces bibliothèques vivent dans des environnements que l'on prépare en début de projet, et leurs versions comptent : un script qui fonctionne chez un camarade peut échouer chez toi si une bibliothèque manque ou diffère, d'où l'habitude professionnelle de noter ses dépendances dans un fichier requirements.txt.

Le Web fonctionne par requêtes HTTP : un client demande une ressource, un serveur répond avec un code de statut (200 = succès, 404 = introuvable, 500 = erreur du serveur) et un contenu, souvent au format JSON. La bibliothèque requests rend cela trivial : reponse = requests.get(url, timeout=5) envoie la requête — le timeout est indispensable pour ne pas rester bloqué sur un serveur muet —, reponse.status_code donne le statut, reponse.json() transforme directement la réponse JSON en dictionnaire ou liste Python, et un bloc try/except requests.RequestException intercepte les pannes réseau. C'est ainsi que les programmes consomment les API : météo, taux de change, catalogues de produits, résultats sportifs publiés en données ouvertes.

pandas est l'outil standard d'analyse de données en Python. Sa structure centrale, le DataFrame, est un tableau avec des lignes et des colonnes nommées, que l'on obtient le plus souvent d'un fichier CSV par pd.read_csv("ventes.csv"). On y ajoute une colonne calculée d'une ligne (df["montant"] = df["quantite"] * df["prix"]), on filtre les lignes par une condition entre crochets, on trie avec sort_values, et on agrège avec groupby(...).sum() pour des totaux par catégorie, le tout sans écrire la moindre boucle. Là où le csv artisanal de la leçon 4 demandait quinze lignes de code, pandas en demande trois — et le code reste lisible pour qui connaît les quatre verbes : lire, filtrer, calculer, regrouper.

L'essentiel
pip installe les bibliothèques de l'écosystème. requests.get + status_code + .json() consomment les API web ; pandas DataFrame lit un CSV et permet filtres, colonnes calculées et regroupements sans boucles.

Exemple — Analyser un fichier de ventes avec pandas

```
import pandas as pd

# ventes.csv contient les colonnes :
# produit, quantite, prix
df = pd.read_csv("ventes.csv")
df["montant"] = df["quantite"] * df["prix"]
print(df)
print("Total général :", df["montant"].sum())
# Total général : 84000
produit_top = df.loc[df["quantite"].idxmax(), "produit"]
print("Produit le plus vendu :", produit_top)
# Produit le plus vendu : sucre

# Et pour interroger un service web :
import requests
try:
r = requests.get("https://api.github.com", timeout=5)
print("Statut HTTP :", r.status_code) # 200
print(r.headers.get("Content-Type"))
# application/json; charset=utf-8
except requests.RequestException as e:
print("Requête impossible :", e)
```

À retenir

- Toujours un timeout sur une requête web : sans lui, un serveur muet bloque ton programme pour toujours.
- Le DataFrame se manipule par colonnes entières : filtrer, calculer et regrouper remplacent les boucles manuelles.
- Une réponse HTTP ne se croit pas sur parole : vérifier status_code avant d'exploiter le contenu.

Exercice

À partir du DataFrame ventes de l'exemple (produits riz, huile, sucre), écris le code pandas qui calcule le montant total des ventes par produit, trié du plus grand au plus petit, et donne le résultat obtenu sur le fichier de l'exemple.

✔ Corrigé
Après avoir créé la colonne montant, une seule ligne suffit : totaux = df.groupby("produit")["montant"].sum().sort_values(ascending=False). groupby regroupe les lignes par produit, sum additionne les montants de chaque groupe, sort_values classe les résultats. Résultat vérifié par exécution : riz 45000, sucre 30000, huile 9000 — total cohérent avec les 84 000 du cours. Le même traitement en Python « à la main » demanderait un dictionnaire accumulateur et une boucle complète : pandas facture cette économie de code sur chaque analyse.

Leçon 7 — Projet : automatiser une tâche réelle avec un script Python

Un script d'automatisation naît toujours de la même observation : une tâche répétitive, faite à la main, qui suit toujours les mêmes règles. Renommer deux cents photos une à une, recopier des notes d'un fichier à l'autre, consolider les listes de présence de la semaine : si un humain peut décrire la procédure sans réfléchir, un programme peut l'exécuter sans se fatiguer — et sans faute d'inattention. La démarche professionnelle comporte quatre temps : observer la tâche manuelle et noter ses règles exactes, découper le traitement en petites fonctions testables, écrire le script en commençant par le cas simple, puis le faire tourner sur un petit échantillon avant le traitement complet.

Prenons un cas réel d'administration : un dossier rempli de fichiers aux noms chaotiques — espaces, majuscules mélangées, accents de frappe — que l'on veut uniformiser en minuscules avec des tirets bas. Le module os fournit listdir pour énumérer les fichiers d'un dossier et rename pour les renommer ; la règle de nettoyage tient en trois opérations de chaîne : strip() pour les espaces de bord, lower() pour les minuscules, replace(" ", "_") pour les espaces internes. Le script affiche chaque renommage au fur et à mesure et un compteur final : un automatisme qui travaille en silence total est un automatisme qu'on ne peut pas contrôler.

Trois habitudes distinguent le script fiable du script dangereux. D'abord, le mode « simulation » : une variable SIMULATION = True qui fait afficher les actions prévues sans les exécuter, pour valider sur le vrai dossier avant de basculer en réel. Ensuite, ne jamais écraser : avant de renommer, vérifier que le nom cible n'existe pas déjà, sinon deux fichiers différents pourraient recevoir le même nom propre et l'un détruirait l'autre. Enfin, traiter les erreurs fichier par fichier avec try/except pour qu'un seul fichier récalcitrant ne bloque pas les cent quatre-vingt-dix-neuf autres, et consigner les échecs dans un petit rapport. Ces réflexes — simuler, protéger, continuer malgré les erreurs — sont exactement ceux des outils professionnels d'administration système.

L'essentiel
Automatiser = décrire une procédure répétitive en règles exactes, la découper en fonctions, tester en simulation, puis exécuter avec garde-fous : pas d'écrasement, erreurs isolées, rapport final.

Exemple — Uniformiser les noms des fichiers d'un dossier

```
import os

dossier = "photos_a_trier"
renommes = 0
for nom in os.listdir(dossier):
propre = nom.strip().lower().replace(" ", "_")
if propre != nom:
ancien = os.path.join(dossier, nom)
nouveau = os.path.join(dossier, propre)
os.rename(ancien, nouveau)
renommes += 1
print("Fichiers renommés :", renommes) # 3
print(sorted(os.listdir(dossier)))
# ['cours_python.pdf', 'photo_vacances.jpg',
# 'rapport_final.pdf']
```

À retenir

• Avant le grand traitement, toujours un essai sur un petit dossier de test ou en mode simulation.

• os.listdir énumère, os.path.join construit les chemins proprement, os.rename déplace ou renomme.

• Un bon script rend compte : actions affichées, compteur final, échecs consignés — jamais de silence.

Exercice

Adapte le script de la leçon pour ne traiter que les fichiers dont le nom se termine par .pdf (quelle que soit la casse d'origine), et afficher à la fin le nombre de PDF renommés ainsi que le nombre de fichiers ignorés. Donne le principe des modifications et le résultat sur le dossier de l'exemple.

✔ Corrigé
On ajoute un test dans la boucle : if not nom.lower().endswith(".pdf"), on incrémente un compteur ignores et on passe au fichier suivant avec continue ; sinon le nettoyage et le renommage se déroulent comme avant, avec leur propre compteur. lower() avant endswith rend le test insensible à la casse, donc « Rapport Final.PDF » est bien reconnu comme un PDF. Résultat vérifié par exécution sur le dossier de l'exemple : 2 PDF renommés (cours_python.pdf, rapport_final.pdf) et 1 fichier ignoré (photo_vacances.jpg, qui garde son nom d'origine).

Travaux dirigés (TD)

TD 1 — Annuaire de contacts persistant en JSON

Construis un petit annuaire téléphonique en Python : un dictionnaire associe chaque nom à un numéro. Écris les fonctions ajouter_contact(annuaire, nom, numero), sauvegarder(annuaire, chemin) qui écrit le dictionnaire dans un fichier JSON (module json, fonction dump), et charger(chemin) qui relit le fichier et retourne le dictionnaire — ou un dictionnaire vide si le fichier n'existe pas encore (FileNotFoundError

interceptée). Programme principal : crée un annuaire, ajoute deux contacts, sauvegarde dans annuaire.json, recharge le fichier dans une nouvelle variable et affiche le résultat pour prouver que les données ont survécu à la « fermeture » du programme.

✔ Corrigé du TD 1
ajouter_contact fait simplement annuaire[nom] = numero. sauvegarder ouvre le fichier en "w" avec encoding="utf-8" et appelle json.dump(annuaire, f, ensure_ascii=False, indent=2) : ensure_ascii=False conserve les accents des noms, indent=2 rend le fichier lisible par un humain. charger met le json.load dans un try et retourne {} dans le except FileNotFoundError : au premier lancement, l'absence de fichier est un cas normal, pas une panne. Résultat vérifié par exécution : après sauvegarde puis rechargement, on obtient {'Maman': '70 00 00 01', 'Oncle Karim': '76 00 00 02'} — exactement les contacts saisis. Le couple JSON + dictionnaire est le format de persistance le plus simple pour les petites applications, et celui des fichiers de configuration de la plupart des outils.

TD 2 — Rapport de résultats d'une classe, généré automatiquement

On dispose des moyennes d'une classe sous forme de liste de couples (nom, moyenne) : [("Awa", 15.5), ("Moussa", 9.0), ("Fatou", 17.0), ("Sekou", 11.5)]. Écris un programme qui : 1) calcule la liste des admis (moyenne ≥ 10) par une compréhension de liste ; 2) calcule le taux de réussite en pourcentage ; 3) écrit un rapport texte dans resultats.txt avec une ligne par étudiant au format « Nom : moyenne » avec deux décimales ; 4) affiche les admis et le taux. Donne les valeurs obtenues.

✔ Corrigé du TD 2
La compréhension [nom for nom, note in resultats if note >= 10] extrait les noms des admis en une ligne : elle donne ['Awa', 'Fatou', 'Sekou']. Le taux vaut len(admis) / len(resultats) * 100, soit 75.0 %. L'écriture du rapport se fait dans un with open("resultats.txt", "w", encoding="utf-8") en bouclant sur les couples et en écrivant f"{nom} : {moy:.2f}\n" — le format :.2f impose deux décimales, donc 15.50 et 9.00 dans le fichier. Résultats vérifiés par exécution, y compris la relecture du fichier produit. Ce mini-programme combine les trois gestes du cours : transformer des données (compréhension), calculer un indicateur (taux), produire un document (fichier formaté).

Travaux pratiques (TP)

TP 1 — Bulletins automatisés à partir d'un fichier CSV de notes

Le secrétariat fournit un fichier notes_classe.csv au format « nom;maths;info » (un étudiant par ligne, avec une ligne d'en-tête). Écris le script complet qui : lit le fichier en sautant l'en-tête ; pour chaque ligne, calcule la moyenne des deux notes et la décision (Admis si moyenne ≥ 10, Ajourné sinon) ; si une note n'est pas un nombre, n'interrompt pas le script mais produit la mention « notes invalides, à vérifier » pour cet étudiant ; affiche tous les bulletins, une ligne par étudiant. Fichier de test :

Awa;14;16 — Moussa;9;11 — Fatou;abc;13 — Sekou;12;8. Donne la sortie exacte attendue et explique le rôle du try/except dans ce traitement.

✔ Attendus du TP 1
Le script ouvre le fichier avec with, consomme l'en-tête par next(f), puis pour chaque ligne : nom, maths, info = ligne.strip().split(";") découpe les trois champs ; le calcul (float(maths) + float(info)) / 2 est placé dans un try, la décision est choisie par un if, et la ligne de bulletin est ajoutée à une liste ; le except ValueError ajoute la ligne « {nom} : notes invalides, à vérifier ». Sortie exacte vérifiée par exécution : « Awa : moyenne 15.00 - Admis », « Moussa : moyenne 10.00 - Admis », « Fatou : notes invalides, a verifier », « Sekou : moyenne 10.00 - Admis ». Le try/except joue ici le rôle d'un contrôle qualité ligne par ligne : une saisie fautive du secrétariat ne doit ni bloquer les bulletins des autres étudiants, ni — pire — produire une moyenne fausse en silence ; elle est signalée nommément pour correction humaine.

Exercices corrigés supplémentaires

Exercice 1

Écris le programme FizzBuzz pour les entiers de 1 à 15 : afficher « FizzBuzz » si le nombre est divisible par 3 et par 5, « Fizz » s'il est divisible par 3 seulement, « Buzz » s'il est divisible par 5 seulement, et le nombre lui-même sinon. Donne la sortie complète.

✔ Corrigé
Boucle for i in range(1, 16) ; on teste d'abord le cas le plus exigeant (i % 15 == 0, qui combine 3 et 5) avant les cas simples, sinon un multiple de 15 serait capturé par le test % 3. Sortie vérifiée par exécution : 1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz. L'ordre des tests dans une cascade est la leçon durable de cet exercice classique : du plus spécifique au plus général.

Exercice 2

À partir de la liste [3, 7, 3, 9, 7, 1, 9], produis la liste des valeurs uniques triées par ordre croissant, en une seule ligne de code. Explique le mécanisme.

✔ Corrigé
uniques = sorted(set(liste)) donne [1, 3, 7, 9], vérifié par exécution. set(liste) convertit la liste en ensemble, ce qui élimine les doublons par construction ; sorted(...) retransforme l'ensemble en liste triée. Deux fonctions standard enchaînées remplacent une boucle de dédoublonnage manuelle — mais souviens-toi que l'ensemble perd l'ordre d'origine : si l'ordre d'apparition comptait, il faudrait une autre méthode.

Exercice 3

Écris une fonction convertir_age(texte) qui convertit le texte en entier, puis lève une ValueError de message « Age invraisemblable » si l'âge obtenu est inférieur à 0 ou supérieur à 120. Montre comment un programme appelant peut intercepter cette erreur et afficher le message.

✔ Corrigé
La fonction fait age = int(texte), teste if age < 0 or age > 120: raise ValueError("Age invraisemblable"), puis retourne age. L'appelant protège l'appel : try: convertir_age("250") except ValueError as e: print("Erreur attrapée :", e) — la variable e reçoit l'exception et son affichage donne le message. Sortie vérifiée : « Erreur attrapée : Age invraisemblable ». Lever ses propres erreurs permet de signaler un problème métier précis plutôt que de retourner une valeur piège comme -1 que l'appelant pourrait oublier de tester.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Que retourne toujours la fonction input() ?

   A) Un entier

   B) Une chaîne de caractères

   C) Un nombre réel

2. Que valent 7 // 2 et 7 % 2 ?

   A) 3 et 1

   B) 3.5 et 1

   C) 4 et -1

3. Laquelle de ces structures est immuable (non modifiable) ?

   A) La liste

   B) Le dictionnaire

   C) Le tuple

4. Quel mot-clé introduit le bloc où l'on place le code susceptible de lever une erreur ?

   A) catch

   B) try

   C) handle

5. Dans une méthode, à quoi sert le premier paramètre, conventionnellement nommé self ?

   A) Il désigne l'objet lui-même, dont on manipule les attributs

   B) Il contient le nom du fichier du programme

   C) Il sert à importer les modules

6. Pour ouvrir un fichier en ajoutant du contenu à la fin sans rien écraser, on utilise le mode…

   A) "r"

   B) "w"

   C) "a"

7. Que vaut [x for x in range(4)] ?

A) [1, 2, 3, 4]

B) [0, 1, 2, 3]

C) [0, 1, 2, 3, 4]

8. Pour lire la valeur d'un dictionnaire d sans risquer d'erreur si la clé manque, on écrit…

A) d.get("cle", valeur_par_defaut)

B) d["cle"] en espérant qu'elle existe

C) d.cle

9. Dans pandas, la structure qui représente un tableau de données avec lignes et colonnes nommées est…

A) La Series unique

B) Le ndarray

C) Le DataFrame

10. Après import math, comment calcule-t-on la racine carrée de 9 ?

A) math.racine(9)

B) math.sqrt(9)

C) sqrt(9) tout court

Corrigés du QCM

1. Réponse B — input() renvoie une str dans tous les cas ; c'est au programme de convertir avec int() ou float() selon le besoin.

2. Réponse A — // est la division entière (quotient 3) et % le reste de la division (1), car 7 = 2 × 3 + 1.

3. Réponse C — Un tuple ne peut être ni agrandi ni modifié après création ; listes et dictionnaires sont modifiables.

4. Réponse B — Python utilise try / except ; catch appartient à d'autres langages comme Java ou JavaScript.

5. Réponse A — self référence l'instance en cours : self.nom accède à l'attribut nom de cet objet précis.

6. Réponse C — "a" (append) ajoute à la fin ; "w" écrase le contenu existant et "r" ne permet que la lecture.

7. Réponse B — range(4) produit 0, 1, 2, 3 : la borne de fin est toujours exclue en Python.

8. Réponse A — get retourne la valeur par défaut fournie quand la clé est absente, au lieu de lever une KeyError.

9. Réponse C — Le DataFrame est le tableau à deux dimensions de pandas ; la Series n'a qu'une seule colonne.

10. Réponse B — On préfixe par le nom du module : math.sqrt(9) vaut 3.0 ; sqrt seul n'existe que si on l'a importé nommément.

Résumé — les notions importantes

• Python mise sur la lisibilité : indentation signifiante, typage dynamique, f-strings pour l'affichage, input() toujours en chaîne.

• Les fonctions (def, return, valeurs par défaut, docstrings) et les modules de la bibliothèque standard structurent les programmes et évitent de réinventer les outils de base.

• Quatre collections complémentaires : liste modifiable, tuple immuable, dictionnaire par clés, ensemble sans doublons — le bon choix dépend de l'usage, pas de l'habitude.

• Les fichiers se manipulent avec with open(...) en précisant mode et encodage ; les formats CSV et JSON ont leurs modules dédiés.

• try/except/else/finally et raise transforment les erreurs en comportements contrôlés au lieu de plantages.

• La POO regroupe attributs et méthodes dans des classes : __init__ initialise, self désigne l'objet, l'héritage spécialise, les méthodes contrôlent les modifications.

• requests consomme les services web (avec timeout et vérification du statut), pandas analyse les tableaux de données ; un script d'automatisation fiable se teste en simulation, protège les données et rend compte de ce qu'il a fait.

Fiches de révision express

Chaque fiche reprend les points « À retenir » d'une leçon. Relis-les la veille d'un contrôle : cinq minutes par fiche suffisent.

Fiche 1 — Syntaxe Python : variables, types et structures de contrôle
L'indentation fait partie de la syntaxe : un mauvais décalage est une erreur ou un bug, jamais un détail esthétique. ; input() renvoie toujours une str : convertir avec int() ou float() avant de calculer. ; for parcourt directement les valeurs d'une collection ; range() sert quand on veut un compteur.

Fiche 2 — Fonctions, modules et bibliothèque standard
Une fonction sans return renvoie None : si ton calcul « disparaît », cherche un return oublié. ; Paramètre avec valeur par défaut = argument facultatif ; l'appel prix_ttc(10000) utilise le taux 0.18. ; import module donne accès à module.fonction ; from module import fonction importe le nom directement.

Fiche 3 — Listes, tuples, dictionnaires et ensembles
Choisir la collection selon le besoin : ordre et modification → liste ; données figées → tuple ; accès par nom → dictionnaire ; unicité → ensemble. ; dict.get(cle, defaut) évite l'erreur KeyError quand la clé peut manquer. ; Un doublon dans un ensemble disparaît silencieusement : parfait pour dédoublonner, dangereux si les répétitions comptaient.

Fiche 4 — Manipulation de fichiers et gestion des erreurs
"w" écrase le fichier existant sans demander : pour conserver l'historique, utiliser "a". ; Attraper l'exception précise (ValueError…) plutôt qu'un except nu qui cacherait tous les bugs. ; Une ligne lue garde son "\n" : penser à strip() avant de convertir ou de comparer.

Fiche 5 — Programmation orientée objet en Python
self n'est pas un mot-clé magique : c'est le premier paramètre de chaque méthode, qui reçoit l'objet courant automatiquement. ; __init__ ne crée pas l'objet, il l'initialise : les attributs naissent là, avec des valeurs cohérentes dès le départ. ; Hériter, c'est réutiliser puis spécialiser : super().__init__ laisse la classe mère faire sa part du travail.

Fiche 6 — Bibliothèques utiles : requests et pandas en initiation
Toujours un timeout sur une requête web : sans lui, un serveur muet bloque ton programme pour toujours. ; Le DataFrame se manipule par colonnes entières : filtrer, calculer et regrouper remplacent les boucles manuelles. ; Une réponse HTTP ne se croit pas sur parole : vérifier status_code avant d'exploiter le contenu.

Fiche 7 — Projet : automatiser une tâche réelle avec un script Python
Avant le grand traitement, toujours un essai sur un petit dossier de test ou en mode simulation. ; os.listdir énumère, os.path.join construit les chemins proprement, os.rename déplace ou renomme. ; Un bon script rend compte : actions affichées, compteur final, échecs consignés — jamais de silence.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — Apprendre Python - Programmation Python pour Débutant

Chaîne : MaxCode · Langue : Français · Niveau : Débutant
Un cours complet qui part de l'installation et couvre variables, types, conditions, boucles, listes, dictionnaires, fonctions et une première approche des objets. Parfait pour accompagner les leçons 1 à 3 en révisant chaque notion en vidéo.

https://www.youtube.com/watch?v=_TVrC_HCkTE

Leçon 3 — Tuple, Set et Dictionnaire en Python

Chaîne : Comment Coder · Langue : Français · Niveau : Débutant
Une vidéo courte et ciblée sur les trois collections souvent confondues : tuples immuables, ensembles sans doublons et dictionnaires clé-valeur. Elle montre concrètement quand choisir chacune d'elles, en complément direct de la leçon 3.

https://www.youtube.com/watch?v=qFiK62vpmUw

Leçon 4 — MEGA TUTO Python : 5 HEURES pour APPRENDRE à CODER ! Les BASES de la PROGRAMMATION avec Python !

Chaîne : Bertrand . Tech · Langue : Français · Niveau : Débutant
Un tutoriel marathon qui enchaîne les bases du langage puis, dans sa dernière partie, la lecture et l'écriture de fichiers ainsi que la gestion des erreurs avec try/except. Idéal pour revoir la leçon 4 avec d'autres exemples commentés pas à pas.

https://www.youtube.com/watch?v=EWtKgJVYQbM

Leçon 5 — APPRENDRE PYTHON : LA PROGRAMMATION ORIENTÉE OBJET [TUTO COMPLET DÉBUTANT]

Chaîne : CodeAvecJonathan · Langue : Français · Niveau : Débutant
La POO expliquée depuis zéro : classes, constructeur, instances, méthodes, variables de classe et héritage, avec des mises en situation et des exercices guidés. C'est le prolongement vidéo naturel de la leçon 5, à suivre code sous les yeux.

https://www.youtube.com/watch?v=Y-wXK0Wu5pc

Leçon 6 — Apprendre Python, Numpy, Pandas pour la Data Science [Cours complet Niveau Débutant avec Notebooks]

Chaîne : J.A DATATECH CONSULTING · Langue : Français · Niveau : Débutant à intermédiaire
Un cours complet qui, après les bases de Python, consacre un long chapitre à pandas : DataFrames, lecture de fichiers et analyses guidées dans des notebooks téléchargeables. À utiliser pour dépasser l'initiation de la leçon 6 et pratiquer sur de vrais jeux de données.

https://www.youtube.com/watch?v=LwkWwxg10IU

Sources et références

• Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.

• Documentation officielle de Python 3 (docs.python.org/fr) : le tutoriel et la référence de la bibliothèque standard, en français.

• Gérard Swinnen — « Apprendre à programmer avec Python 3 » (Eyrolles) : ouvrage de référence francophone, également diffusé librement par son auteur.

• Documentation officielle de pandas (pandas.pydata.org) et de requests (requests.readthedocs.io) pour aller au-delà de l'initiation de la leçon 6.

• Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés à la publication).

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !

Mes notes

Utilise ces pages pour recopier les définitions, refaire les calculs et noter tes questions pour le prochain cours.